

# Chap 4. Data Types

# Contents

- 01 Data Type**
- 02 Characters**
- 03 Numbers**
- 04 Decimal Precision**
- 05 Memory Size**
- 06 Data Conversion**
- 07 Constants**

# Basic Data Types

## ■ Basic Data Types

- The data type specifies the size and type of information the variable will store.

| Data Type | Size         | Description                                                                                           | Example |
|-----------|--------------|-------------------------------------------------------------------------------------------------------|---------|
| int       | 2 or 4 bytes | Stores whole numbers, without decimals                                                                | 1       |
| float     | 4 bytes      | Stores fractional numbers, containing one or more decimals. Sufficient for storing 6-7 decimal digits | 1.99    |
| double    | 8 bytes      | Stores fractional numbers, containing one or more decimals. Sufficient for storing 15 decimal digits  | 1.99    |
| char      | 1 byte       | Stores a single character/letter/number, or ASCII values                                              | 'A'     |

## ■ Basic Format Specifiers

| Format Specifier | Data Type                               |
|------------------|-----------------------------------------|
| %d or %i         | int                                     |
| %f or %F         | float                                   |
| %lf              | double                                  |
| %c               | char                                    |
| %s               | Used for <a href="#">strings</a> (text) |

# Character Data Types

## ■ The char Type

- The char data type is used to store a single character.
- The character must be surrounded by single quotes, like 'A' or 'c', and we use the %c format specifier to print it:

```
1 char myGrade = 'A';  
2 printf("%c", myGrade);  
3  
4 char a = 65, b = 66, c = 67;  
5 printf("%c", a);  
6 printf("%c", b);  
7 printf("%c", c);  
8
```

- To store multiple characters (or whole words), use [strings](#) (which you will learn more about in a later chapter):

```
1 char myText[] = "Hello";  
2 printf("%s", myText);
```

# Numeric Data Types

## ■ Numeric

```
1 int myNum = 1000;
2 printf("%d", myNum);
3
4 float myNum = 5.75;
5 printf("%f", myNum);
6
7 double myNum = 19.99;
8 printf("%lf", myNum);
9
10 float f1 = 35e3;    // 35 * 10^3 = 35000
11 double d1 = 12E4;  // 12 * 10^4 = 120000
12
13 printf("%f\n", f1);
14 printf("%lf", d1);
```

## ■ float vs. double

- The **precision** of a floating point value indicates how many digits the value can have after the decimal point.
- The precision of float is six or seven decimal digits
- double variables have a precision of about 15 digits

# Numeric Data Types

## ■ Decimal Precision

- if you print a floating point number, the output will show many digits after the decimal point:
- If you want to remove the extra zeros (set decimal precision), you can use a dot (.) followed by a number that specifies how many digits that should be shown after the decimal point:

```
1 float myFloatNum = 3.5;
2 double myDoubleNum = 19.99;
3
4 printf("%f\n", myFloatNum); // Outputs 3.500000
5 printf("%lf", myDoubleNum); // Outputs 19.990000
6
7 float myFloatNum = 3.5;
8
9 printf("%f\n", myFloatNum); // Default will show 6 digits after the decimal point
10 printf("%.1f\n", myFloatNum); // Only show 1 digit
11 printf("%.2f\n", myFloatNum); // Only show 2 digits
12 printf("%.4f", myFloatNum); // Only show 4 digits
```

# The sizeof Operator

## ■ Get the Memory Size

| Data Type | Size         |
|-----------|--------------|
| int       | 2 or 4 bytes |
| float     | 4 bytes      |
| double    | 8 bytes      |
| char      | 1 byte       |

- To actually get the size (in bytes) of a data type or variable, use the sizeof operator:

```
1 int myInt;  
2 float myFloat;  
3 double myDouble;  
4 char myChar;  
5  
6 printf("%zu\n", sizeof(myInt));  
7 printf("%zu\n", sizeof(myFloat));  
8 printf("%zu\n", sizeof(myDouble));  
9 printf("%zu\n", sizeof(myChar));
```

# Extended Types

## ■ More Data Types

| Type                   | Size*              | Range (commonly)                                                                                               | Format Specifier |
|------------------------|--------------------|----------------------------------------------------------------------------------------------------------------|------------------|
| short int              | 2 bytes            | -32,768 to 32,767                                                                                              | %hd              |
| unsigned int           | 2 or 4 bytes       | 0 to 65,535 (2 bytes)<br>0 to 4,294,967,295 (4 bytes)                                                          | %u               |
| long int               | 4 or 8 bytes       | -2,147,483,648 to 2,147,483,647 (4 bytes)<br>-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807 (8 bytes) | %ld              |
| long long int          | 8 bytes            | -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807                                                        | %lld             |
| unsigned long int      | 4 or 8 bytes       | 0 to 4,294,967,295 (4 bytes)<br>0 to 18,446,744,073,709,551,615 (8 bytes)                                      | %lu              |
| unsigned long long int | 8 bytes            | 0 to 18,446,744,073,709,551,615                                                                                | %llu             |
| long double            | 8, 12, or 16 bytes | Implementation-dependent, but more precision than double                                                       | %Lf              |



# Extended Types

## ■ Example Code

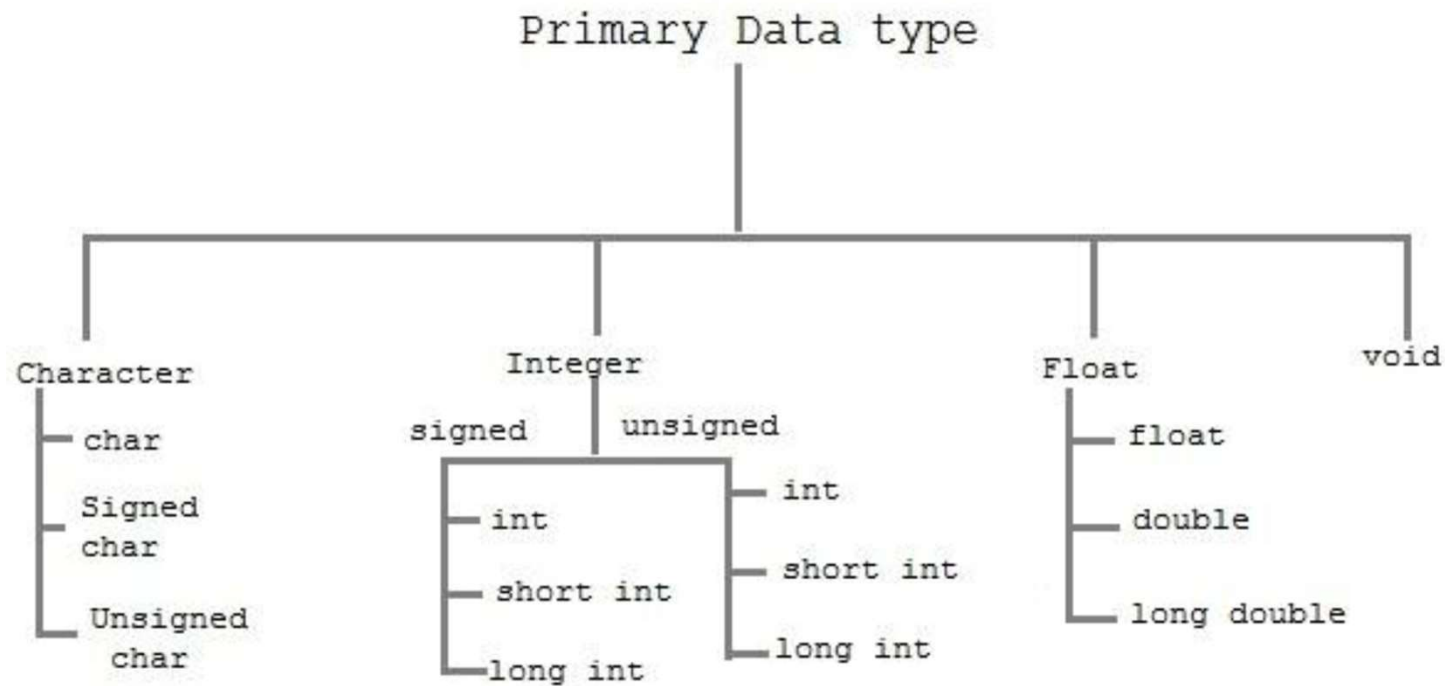
```
1 int normalInt = 1000;           // standard int
2 double normalDouble = 3.14;    // standard double
3
4 short int small = -100;        // smaller int
5 unsigned int count = 25;       // only positive int
6 long int big = 1234567890;     // larger int
7 long long int veryBig = 9223372036854775807; // very large int
8 unsigned long long int huge = 18446744073709551615U; // very large, only positive
9 long double precise = 3.141592653589793238L; // extended precision
10
11 printf("Normal int: %d\n", normalInt);
12 printf("Normal double: %lf\n", normalDouble);
13 printf("Small: %hd\n", small);
14 printf("Count: %u\n", count);
15 printf("Big: %ld\n", big);
16 printf("Very Big: %lld\n", veryBig);
17 printf("Huge: %llu\n", huge);
18 printf("Precise: %Lf\n", precise);
```

# Extended Types

## ■ Check size

```
1 printf("Size of int: %zu bytes\n", sizeof(int));
2 printf("Size of double: %zu bytes\n", sizeof(double));
3 printf("Size of short int: %zu bytes\n", sizeof(short int));
4 printf("Size of unsigned int: %zu bytes\n", sizeof(unsigned int));
5 printf("Size of long int: %zu bytes\n", sizeof(long int));
6 printf("Size of long long int: %zu bytes\n", sizeof(long long int));
7 printf("Size of unsigned long long int: %zu bytes\n", sizeof(unsigned long long int));
8 printf("Size of long double: %zu bytes\n", sizeof(long double));
```

# Data type



# Code Challenge

## ■ Create a Student Report Card

### ■ Instructions

- Inside main(), complete the following steps:
  1. Declare an int named studentID and assign it a value
  2. Declare a float named score and assign it a value (with decimals)
  3. Declare a char named grade and assign it a single letter (in single quotes)
  4. Print all three values using printf (use the correct format specifiers).

```
1 #include <stdio.h>
2
3 int main() {
4     // Write studentID here (1001)
5     // Write score here ( 92.5)
6     // Write grade here ('A')
7
8     // Print studentID
9     // Print score
10    // Print grade
11
12    return 0;
13 }
```

```
1 #include <stdio.h>
2
3 int main() {
4     int studentID = 1001;
5
6
7
8
9
10
11
12    return 0;
13 }
14
```

# Data Conversion

## ■ type conversion

- convert the value of one data type to another type

```
1 int x = 5;  
2 int y = 2;  
3 int sum = 5 / 2;  
4  
5 printf("%d", sum); // Outputs 2  
6  
7 x = 1;  
8 y = 2;  
9 sum = x / y; // Outputs 0
```

- To get the right result, you need to know how type conversion works.
- There are two types of conversion in C:
  - *Implicit Conversion* (automatically)
  - *Explicit Conversion* (manually)

# Data Conversion

## ■ Implicit Conversion

- Implicit conversion is done *automatically by the compiler* when you assign a value of one type to another.

```
1 // Automatic conversion: int to float
2 float myFloat = 9;
3
4 printf("%f", myFloat); // 9.000000
5
6 // Automatic conversion: float to int
7 int myInt = 9.99;
8
9 printf("%d", myInt); // 9
10
11 float sum = 5 / 2;
12
13 printf("%f", sum); // 2.000000
```

# Data Conversion

## ■ Explicit Conversion

- Explicit conversion is done *manually by placing the type in parentheses ()* in front of the value.

```
1 // Manual conversion: int to float
2 float sum = (float) 5 / 2;
3
4 printf("%f", sum); // 2.500000
5
6 int num1 = 5;
7 int num2 = 2;
8 float sum = (float) num1 / num2;
9
10 printf("%f", sum); // 2.500000
11
12 int num1 = 5;
13 int num2 = 2;
14 float sum = (float) num1 / num2;
15
16 printf("%.1f", sum); // 2.5
```

# Constants

## ■ Constants

- unchanging value
- This can be done with the *const* keyword, which makes a variable unchangeable and read-only:

```
1 const int myNum = 15; // myNum will always be 15
2 myNum = 10; // error: assignment of read-only variable 'myNum'
3
4 const int minutesPerHour = 60;
5 const int monthsInYear = 12;
```

- Another thing about constant variables, is that it is considered good practice to declare them with uppercase.
- It is not required, but useful for code readability and common for C programmers:

```
1 const int BIRTHYEAR = 1980;
2 const double PI = 3.141592;
3
```



# Thank You !